

Capture the Flag

Last mission, you built reusable Angular components for the Student Portal.

Unfortunately, you hardcoded every piece of information directly into the HTML. Every student's profile looks identical, and announcements can only be changed by editing the HTML.

Your mission is to **bring the portal to life using Angular Interpolation** (`{{ }}`).

Note from the instructor before you start:

These activities involve using your own logic. We are strictly to implement NOT TO USE AI to generate the logic for you. You may only use AI for RESEARCHING the syntax or UI, but NEVER THE LOGIC.

Consequence: Immediate 0 score in your grade for this activity.

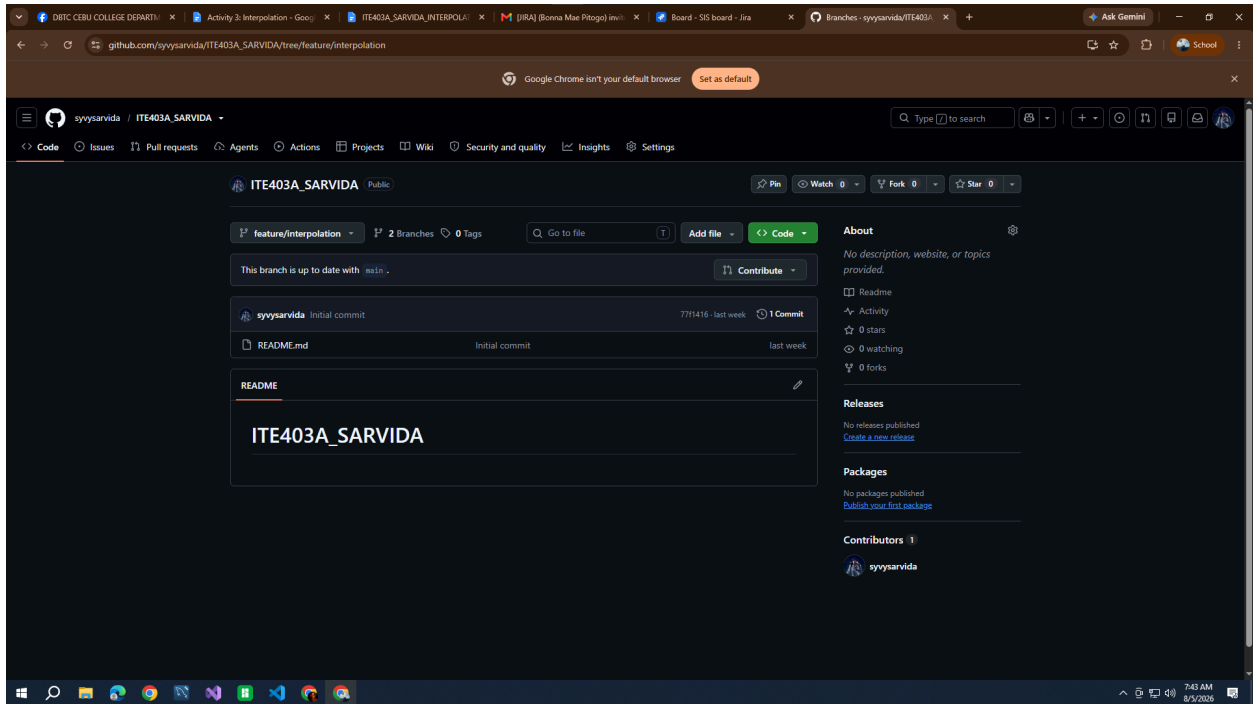
Flag 0 – Mission Branch

Create and switch to a new branch: `feature/interpolation`



Capture

- Show your current Git branch.



Flag 1 – Escape the Hardcoded Trap

Open your **Student Profile Card**.

Right now, your HTML probably looks something like this:

```
<h2>Juan Dela Cruz</h2>
<p>BSIT</p>
<p>3rd Year</p>
<p>2026-0001</p>
```

Your mission is to remove the hardcoded values.

Move them into your **TypeScript** component instead. (".ts")

Hint

```
{{ variableName }}
```

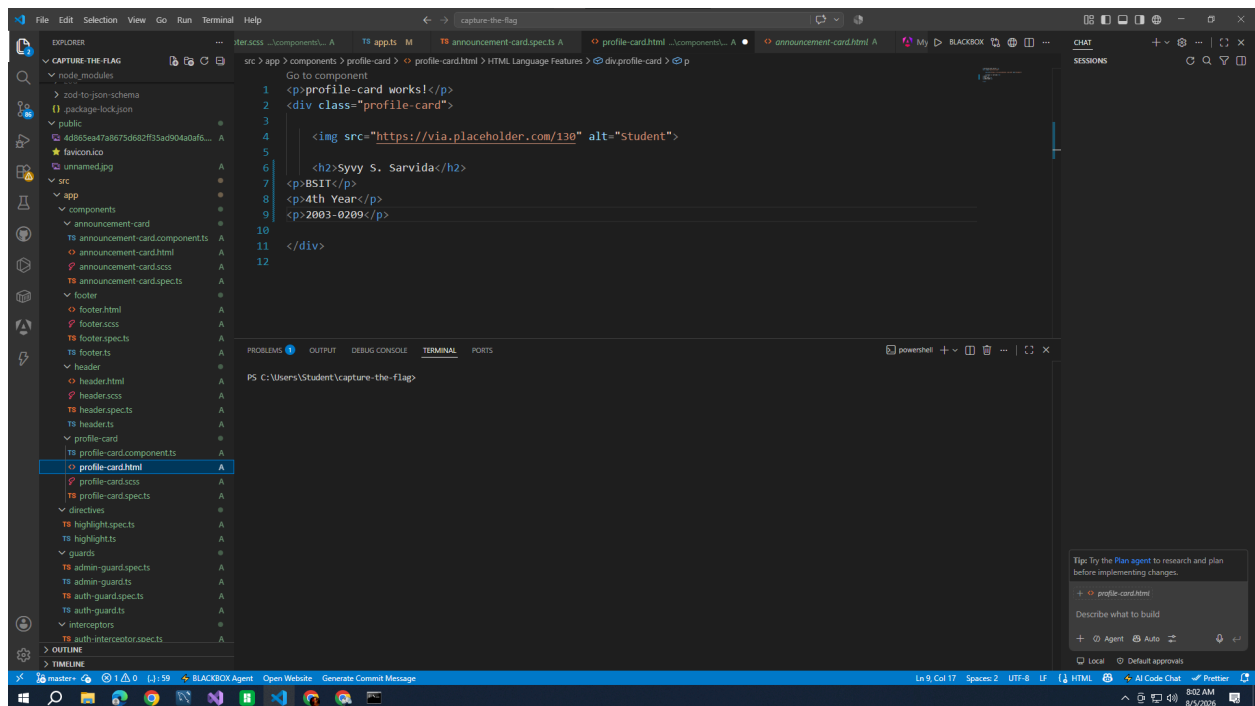
Your HTML should no longer contain actual student information. It should only reference variables.

Your profile card should display:

- Student Name
- Student Number
- Course
- Year Level
- Section
- Nickname (all lowercase)
- Email
- Grade

Capture

- HTML file
- TypeScript file
- Browser



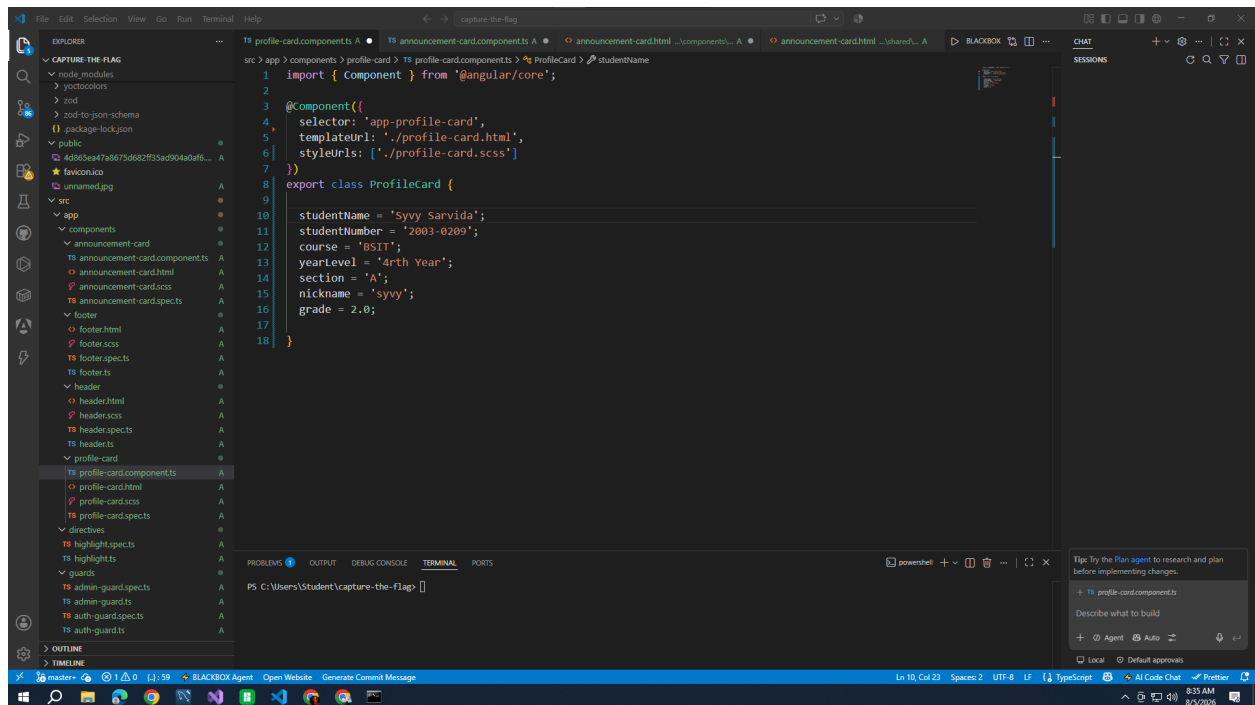
Flag 2 – Dynamic Profile Card

Verify that **every** piece of information in your profile card now comes from TypeScript.

If changing a value in the TypeScript file updates the browser without touching the HTML, you've captured the flag.

Capture

- Updated TypeScript file with changed value
- Browser



Flag 3 – Dynamic Announcements

Your announcement component is also hardcoded.

Instead of writing directly in HTML, create variables for each announcement.

Example idea:

announcementTitle1
announcementBody1

announcementTitle2

announcementBody2

announcementTitle3

announcementBody3

Use interpolation so every announcement card gets its content from TypeScript.



Imagine these values came from a JSON file or API in the future.



Capture

- HTML file
- TypeScript file
- Browser

```
1 <div class="announcement-card">
2   <h3>{{ announcementTitle1 }}</h3>
3   <p>{{ announcementBody1 }}</p>
4 </div>
5
6 <div class="announcement-card">
7   <h3>{{ announcementTitle2 }}</h3>
8   <p>{{ announcementBody2 }}</p>
9 </div>
10
11 <div class="announcement-card">
12   <h3>{{ announcementTitle3 }}</h3>
13   <p>{{ announcementBody3 }}</p>
14 </div>
```

Flag 4 – Thinking Like Data

Now add 7 more cards to make the announcements a total of 10 cards.



Capture

- Show your ten announcement cards in Browser
- TypeScript file

Notice how your announcement variables are starting to repeat.

This is normal, but not ideal.

Question to think about (you don't need to answer, just think):

If there were 100 announcements, would creating 200 variables be a good idea?

(You'll solve this in a future lesson.)

```
16 <div class="announcement-card">
17   <h3>{{ announcementTitle4 }}</h3>
18   <p>{{ announcementBody4 }}</p>
19 </div>
20
21 <div class="announcement-card">
22   <h3>{{ announcementTitle5 }}</h3>
23   <p>{{ announcementBody5 }}</p>
24 </div>
25
26 <div class="announcement-card">
27   <h3>{{ announcementTitle6 }}</h3>
28   <p>{{ announcementBody6 }}</p>
29 </div>
30
31 <div class="announcement-card">
32   <h3>{{ announcementTitle7 }}</h3>
33   <p>{{ announcementBody7 }}</p>
34 </div>
35
36 <div class="announcement-card">
37   <h3>{{ announcementTitle8 }}</h3>
38   <p>{{ announcementBody8 }}</p>
39 </div>
40
41 <div class="announcement-card">
42   <h3>{{ announcementTitle9 }}</h3>
43   <p>{{ announcementBody9 }}</p>
44 </div>
45
46 <div class="announcement-card">
47   <h3>{{ announcementTitle10 }}</h3>
48   <p>{{ announcementBody10 }}</p>
49 </div>
```

Flag 5 – Academic Standing

Your profile already displays the student's grade.

Now make the portal smarter.

Below the grade, display:

- **Pass**
- **Fail**

depending on the grade.

 **Hint**

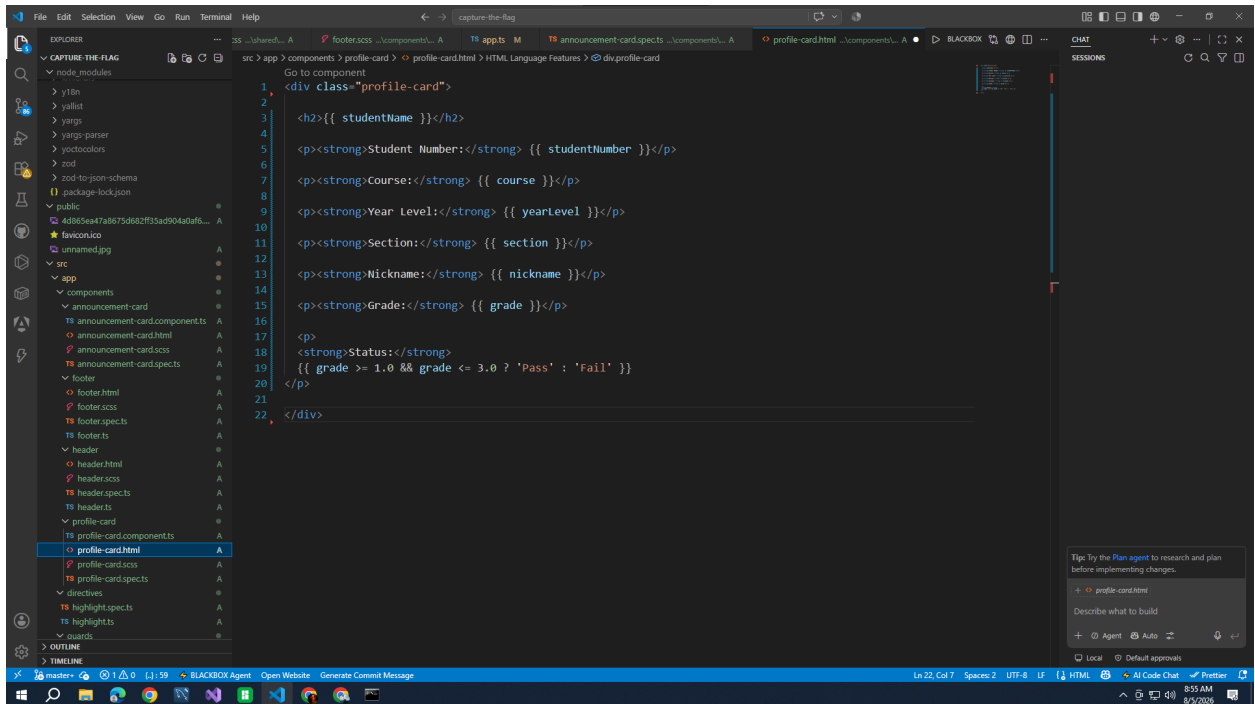
Angular interpolation can evaluate expressions.

```
{{ condition ? "True Value" : "False Value" }}
```

Grading condition: A student can only pass if their grade is 1.0 to 3.0.

Capture

- Interpolation condition
- Browser



```
1 <div class="profile-card">
2
3   <h2>{{ studentName }}</h2>
4
5   <p><strong>Student Number:</strong> {{ studentNumber }}</p>
6
7   <p><strong>Course:</strong> {{ course }}</p>
8
9   <p><strong>Year Level:</strong> {{ yearLevel }}</p>
10
11  <p><strong>Section:</strong> {{ section }}</p>
12
13  <p><strong>Nickname:</strong> {{ nickname }}</p>
14
15  <p><strong>Grade:</strong> {{ grade }}</p>
16
17  <p>
18    <strong>Status:</strong>
19    {{ grade >= 1.0 && grade <= 3.0 ? 'Pass' : 'Fail' }}
20  </p>
21
22 </div>
```

Flag 6 – Formatting Text

Your student's nickname is stored in lowercase.

Example:

juan

Display it in uppercase without changing the value of the variable.

Hint

Interpolation can call simple methods.

Expected:

JUAN

Capture

- Method used to display it in uppercase
- Browser

```
17 <p>
18 <strong>Nickname:</strong>
19 {{ nickname.toUpperCase() }}
20 </p>
21
22 <p>
23 <strong>Status:</strong>
24 {{ grade >= 1.0 && grade <= 3.0 ? 'Pass' : 'Fail' }}
25 </p>
26
27 <p>
28 <strong>Email:</strong>
29
30 {{
31     studentName
32     .toLowerCase()
33     .replace(searchValue: ' ', replaceValue: '.')
34     + '@dbtc-cebu.edu.ph'
35 }}
36 </p>
37
38 </div>
```

Flag 7 – School Email

Every student has the same format of email in Don Bosco.

Example:

juan.delacruz@dbtc-cebu.edu.ph

Instead of storing the full email, remove the “email” variable, then construct the email inside the template, by using the student’s name as variable, concatenated with the email suffix.

No extra variable should be created.

Capture

- Your logic to concatenate the name and email suffix using interpolation
- Browser

```
27 <p>
28   <strong>Email:</strong>
29
30   {{
31     studentName
32       .toLowerCase()
33       .replace(searchValue: ' ', replaceValue: '.')
34     + '@dbtc-cebu.edu.ph'
35   }}
36 </p>
37
38 </div>
```

Flag 8 – Copyright Footer

Every website needs a footer.

Create a variable:

`currentYear`

Display:

© 2026 Student Portal

using interpolation.

Example idea:

© {{ currentYear }}

Capture

- Method used to display the currentYear without hardcoding “2026”
- Browser

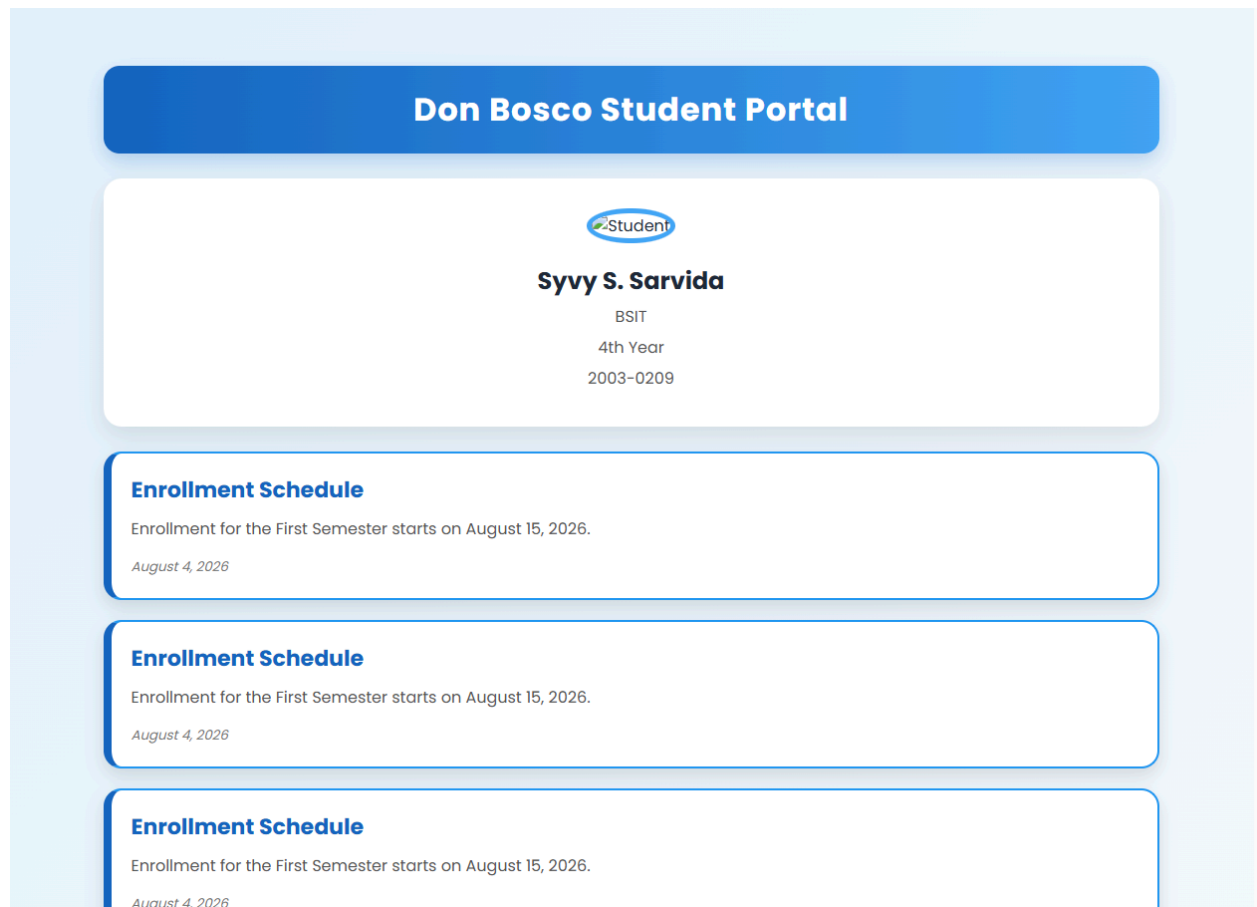
```
1 <footer>
2
3   © {{ currentYear }} Student Portal
4
5 </footer>
```

Imagine the year changes from 2026 to 2027. If you hardcoded the year, you will have to change this value yearly. You need to have a function for that.

Flag 9 – Commit Your Progress

Final Flag Screenshot

- The entire completed browser UI for this activity
- Successful push to `feature/interpolation`



Enrollment Schedule

Enrollment for the First Semester starts on August 15, 2026.

August 4, 2026

Enrollment Schedule

Enrollment for the First Semester starts on August 15, 2026.

August 4, 2026

Enrollment Schedule

Enrollment for the First Semester starts on August 15, 2026.

August 4, 2026

Enrollment Schedule

Enrollment for the First Semester starts on August 15, 2026.

August 4, 2026

Enrollment Schedule

Enrollment for the First Semester starts on August 15, 2026.

August 4, 2026

Enrollment Schedule

Enrollment for the First Semester starts on August 15, 2026.

August 4, 2026

Enrollment Schedule

Enrollment for the First Semester starts on August 15, 2026.

August 4, 2026

